

Отчет по лабораторной работе №5

Компьютерный практикум по статистическому анализу данных

Амуничников Антон Игоревич

Содержание

1	Цель работы	6
2	Задание	7
3	Теоретическое введение	8
4	Выполнение лабораторной работы	9
4.1	Выполнение примеров	9
4.2	Задания для самостоятельной работы	37
5	Выводы	44
	Список литературы	45

Список иллюстраций

4.1 Основные пакеты для работы с графиками в Julia	10
4.2 Основные пакеты для работы с графиками в Julia	11
4.3 Основные пакеты для работы с графиками в Julia	11
4.4 Основные пакеты для работы с графиками в Julia	12
4.5 Основные пакеты для работы с графиками в Julia	12
4.6 Опции при построении графика	13
4.7 Опции при построении графика	14
4.8 Опции при построении графика	15
4.9 Простой точечный график	16
4.10 Точечный график с кодированием значения размером точки . . .	17
4.11 3-мерный точечный график с кодированием значения размером точки	18
4.12 Аппроксимация данных	19
4.13 Две оси координат	20
4.14 Полярные координаты	21
4.15 Параметрический график кривой на плоскости	21
4.16 Параметрический график кривой в пространстве	22
4.17 Параметрический график кривой в пространстве	23
4.18 Линии уровня	24
4.19 Линии уровня	25
4.20 Векторные поля	26
4.21 Gif-анимация	27
4.22 Гипоциклоида	28
4.23 Гипоциклоида	29
4.24 Гипоциклоида	30
4.25 Errorbars	31
4.26 Errorbars	32
4.27 Errorbars	33
4.28 Использование пакета Distributions	34
4.29 Подграфики	35
4.30 Подграфики	35
4.31 Подграфики	36
4.32 Подграфики	37
4.33 Подграфики	37
4.34 Задание №1	38
4.35 Задание №2	38

4.36 Задание №3	38
4.37 Задание №4	39
4.38 Задание №5	39
4.39 Задание №6	40
4.40 Задание №7	40
4.41 Задание №8	41
4.42 Задание №9	41
4.43 Задание №10	42
4.44 Задание №11	43

Список таблиц

1 Цель работы

Основная цель работы – освоить синтаксис языка Julia для построения графиков.

2 Задание

1. Используя JupyterLab, повторите примеры. При этом дополните графики обозначениями осей координат, легендой с названиями траекторий, названиями графиков и т.п.
2. Выполните задания для самостоятельной работы.

3 Теоретическое введение

Julia – высокоуровневый свободный язык программирования с динамической типизацией, созданный для математических вычислений **[julialang]**. Эффективен также и для написания программ общего назначения. Синтаксис языка схож с синтаксисом других математических языков, однако имеет некоторые существенные отличия.

Для выполнения заданий была использована официальная документация Julia **[juliadoc]**.

4 Выполнение лабораторной работы

4.1 Выполнение примеров

Выполним примеры из лабораторной работы для знакомства с пакетами по отрисовки графиков и их функциями (рис. 4.1 - рис. 4.33).

```
# задание функции:
f(x) = (3x.^2 + 6x .- 9).*exp.(-0.3x)

f (generic function with 1 method)

# генерирование массива значений x в диапазоне от -5 до 10 с шагом 0,1
# (шаг задан через указание длины массива):
x = collect(range(-5,10,length=151))

... 151-element Vector{Float64}:
 -5.0
 -4.9
 -4.8
 -4.7
 -4.6
 -4.5
 -4.4
 -4.3
 -4.2
 -4.1
 -4.0
 -3.9
 -3.8
  ⋮
  8.9
  9.0
  9.1
  9.2
  9.3
  9.4
  9.5
  9.6
  9.7
  9.8
  9.9
 10.0

# генерирование массива значений y:
y = f(x)

151-element Vector{Float64}:
161.34080653217032
146.26477779394003
132.19219298833204
119.06942359634911
106.8453557470588
 95.47128188475011
```

Рисунок 4.1: Основные пакеты для работы с графиками в Julia

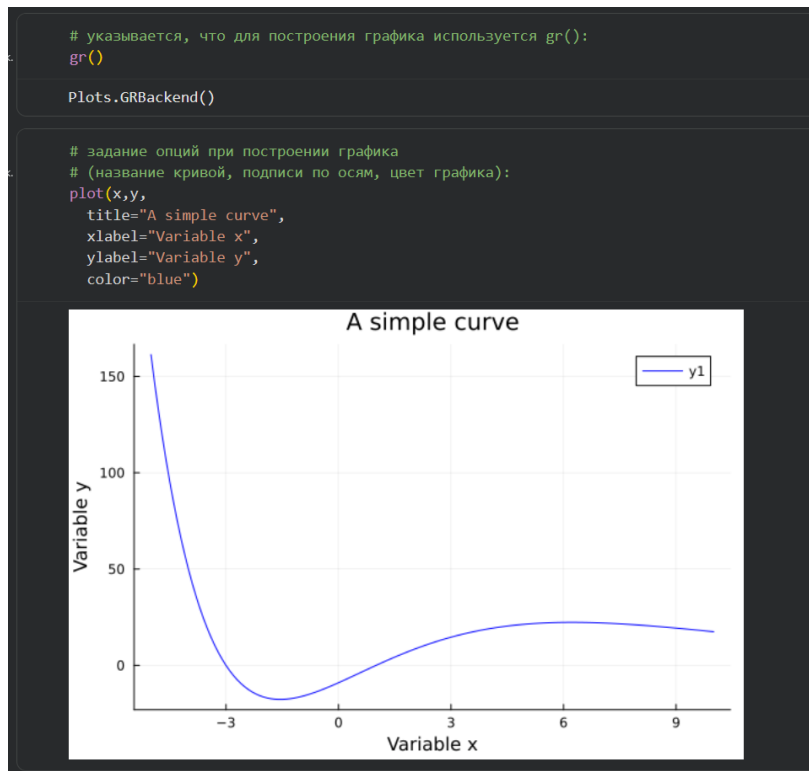


Рисунок 4.2: Основные пакеты для работы с графиками в Julia

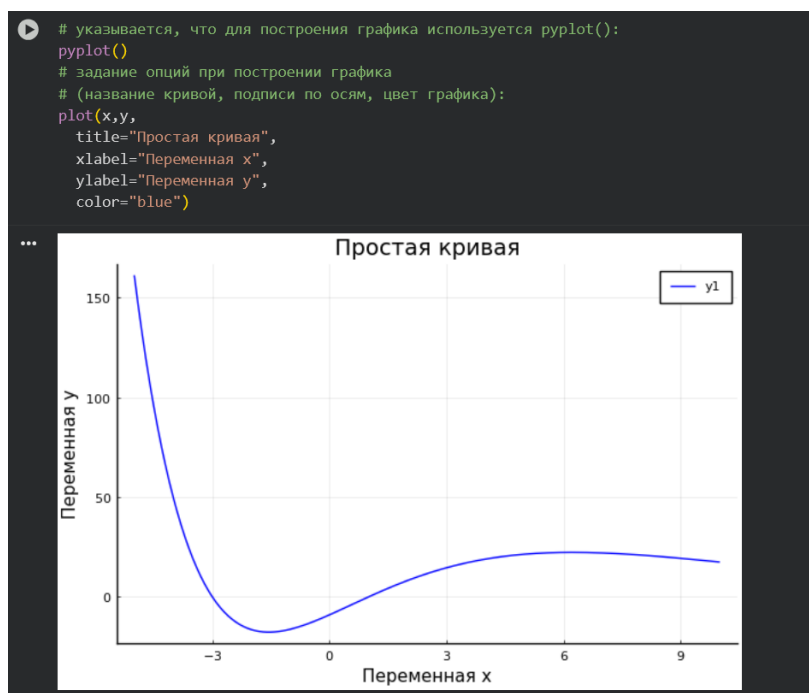


Рисунок 4.3: Основные пакеты для работы с графиками в Julia

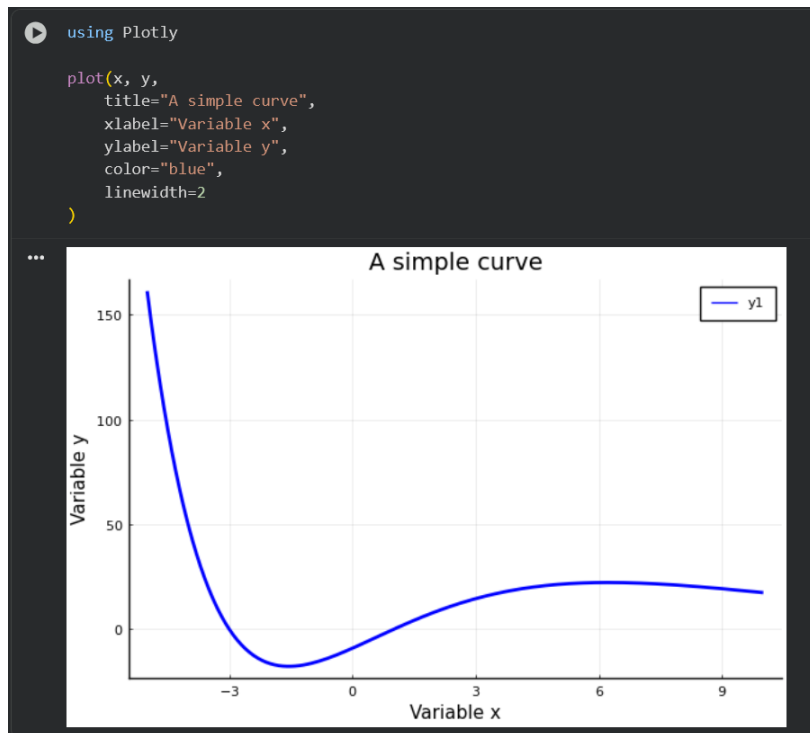


Рисунок 4.4: Основные пакеты для работы с графиками в Julia

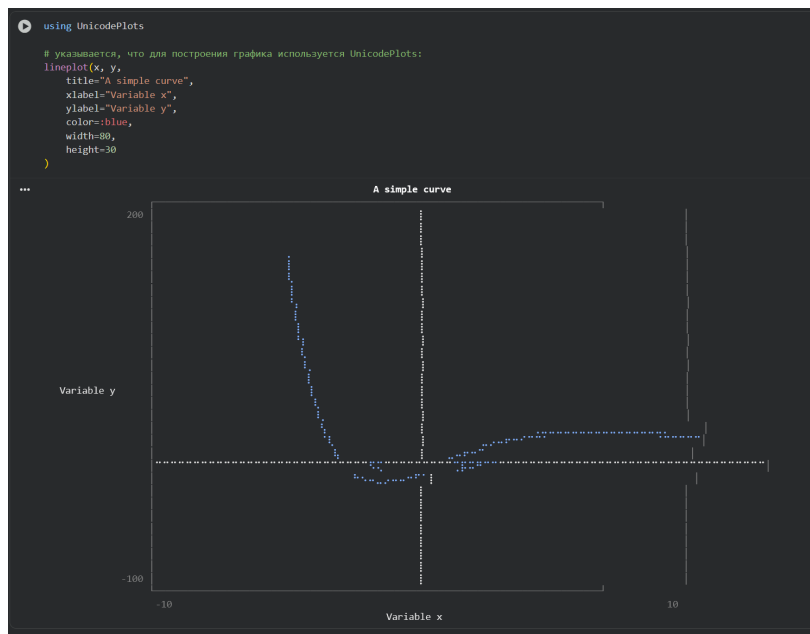
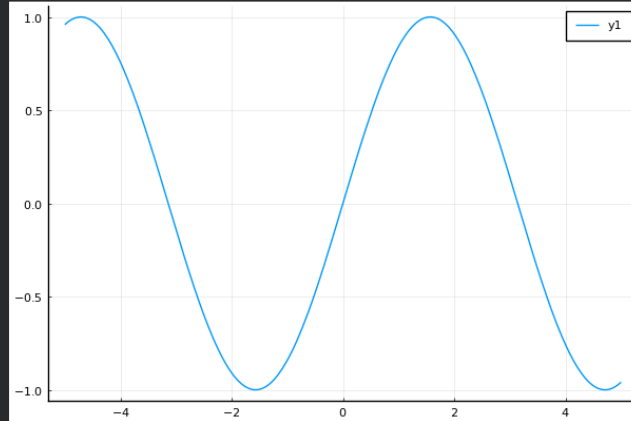


Рисунок 4.5: Основные пакеты для работы с графиками в Julia

5.2.2. Опции при построении графика

```
# указывается, что для построения графика используется pyplot():
pyplot()
# задание функции sin(x):
sin_theor(x) = sin(x)
# построение графика функции sin(x):
plot(sin_theor)
```



```
# задание функции разложения исходной функции в ряд Тейлора:
sin_taylor(x) = [(-1)^i * x^(2*i+1) / factorial(2*i+1) for i in 0:4] |> sum
# построение графика функции sin_taylor(x):
plot(sin_taylor)
```

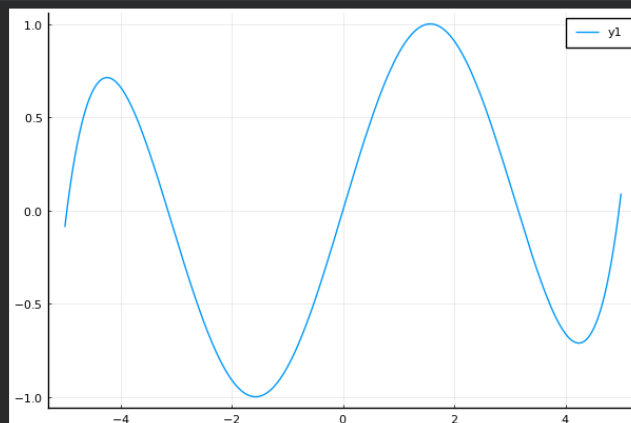


Рисунок 4.6: Опции при построении графика

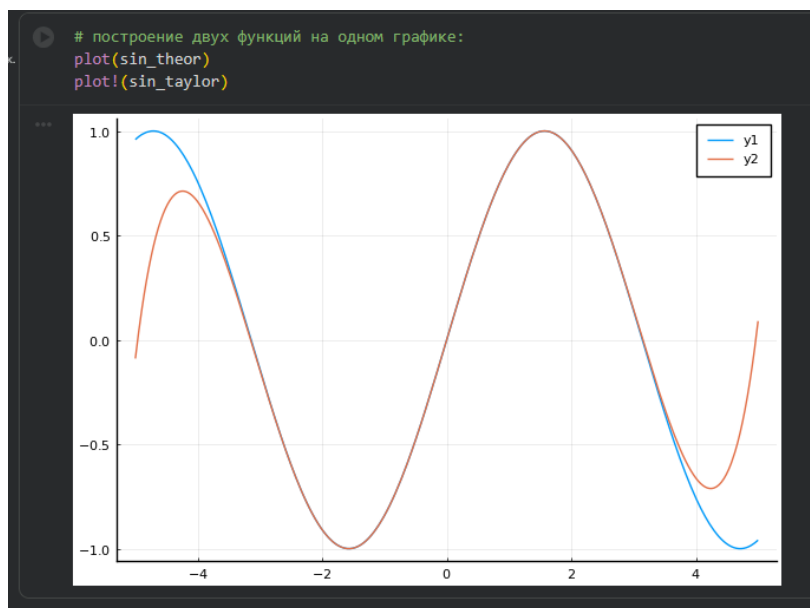


Рисунок 4.7: Опции при построении графика

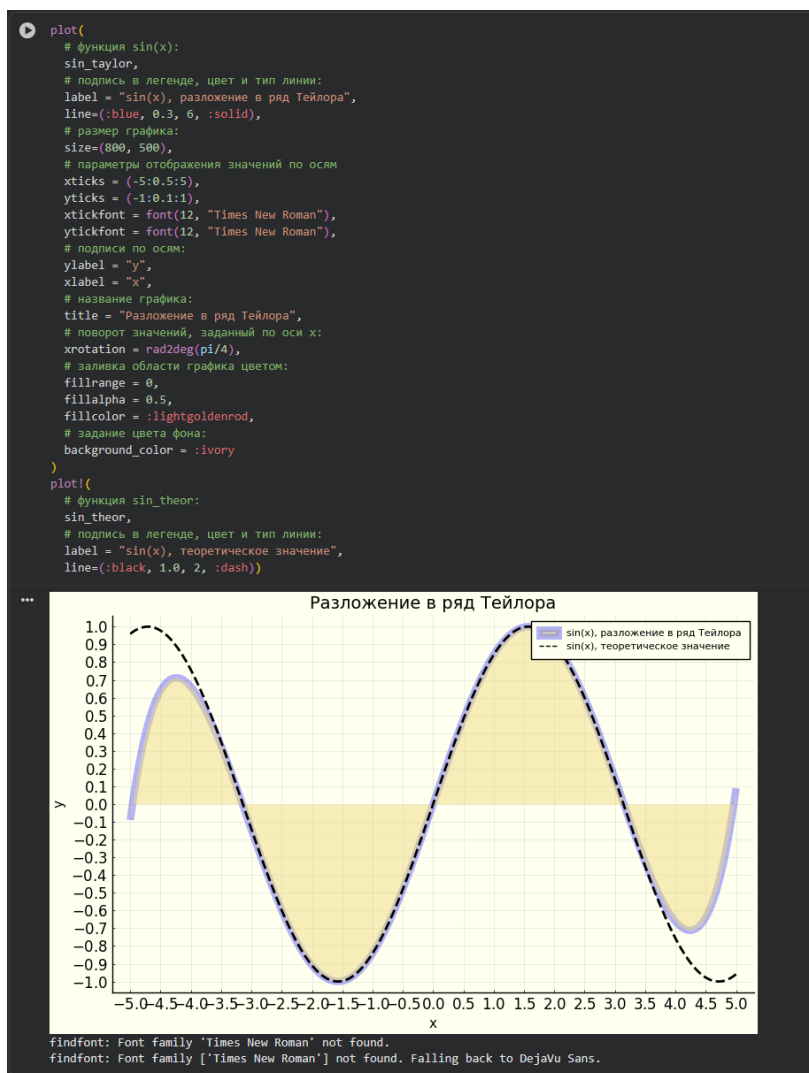


Рисунок 4.8: Опции при построении графика

5.2.3.1. Простой точечный график

```
# параметры распределения точек на плоскости:  
x = range(1,10,length=10)  
y = rand(10)  
# параметры построения графика:  
plot(x, y,  
      seriestype = :scatter,  
      title = "Точечный график"  
)
```

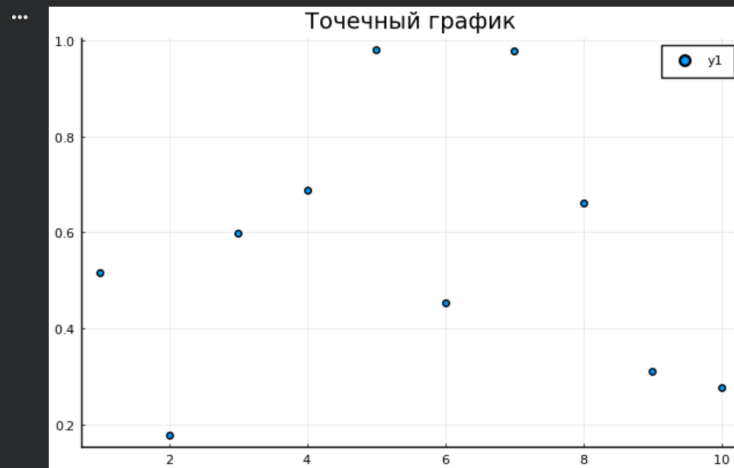


Рисунок 4.9: Простой точечный график

5.2.3.2. Точечный график с кодированием значения размером точки

```
# параметры распределения точек на плоскости:  
n = 50  
x = rand(n)  
y = rand(n)  
ms = rand(50) * 30  
# параметры построения графика:  
scatter(x, y, markersize=ms)
```

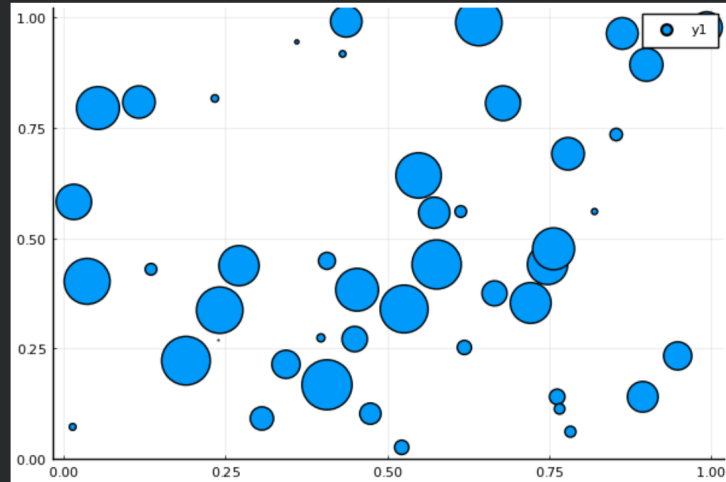


Рисунок 4.10: Точечный график с кодированием значения размером точки

5.2.3.3. 3-мерный точечный график с кодированием значения размером точки

```
# параметры распределения точек в пространстве:  
n = 50  
x = rand(n)  
y = rand(n)  
z = rand(n)  
ms = rand(50) * 30  
# параметры построения графика:  
scatter(x, y, z, markersize=ms)
```

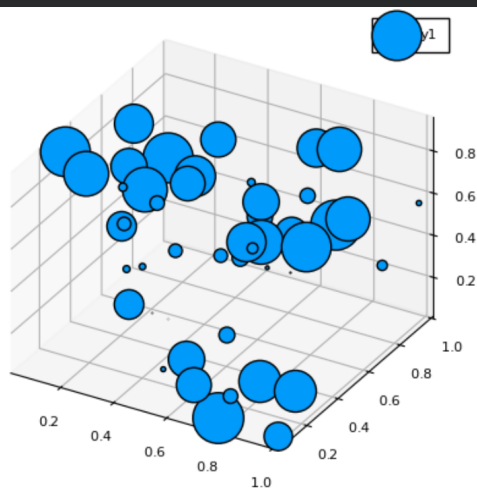


Рисунок 4.11: 3-мерный точечный график с кодированием значения размером точки



Рисунок 4.12: Аппроксимация данных

5.2.5. Две оси ординат

```
# пример случайной траектории
# (заданы обозначение траектории, легенда вверху справа, без сетки)
plot(randn(100),
     ylabel="y1",
     leg=:topright,
     grid = :off,
)
```

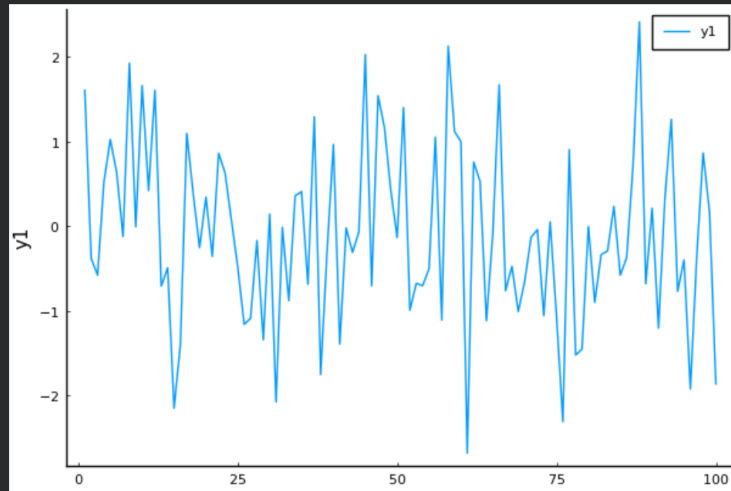


Рисунок 4.13: Две оси координат

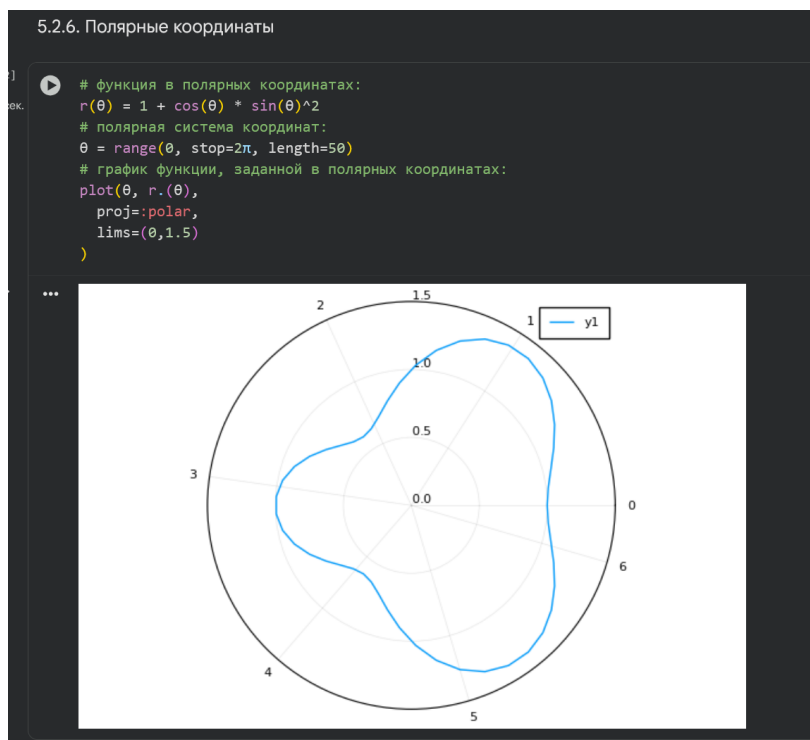


Рисунок 4.14: Полярные координаты

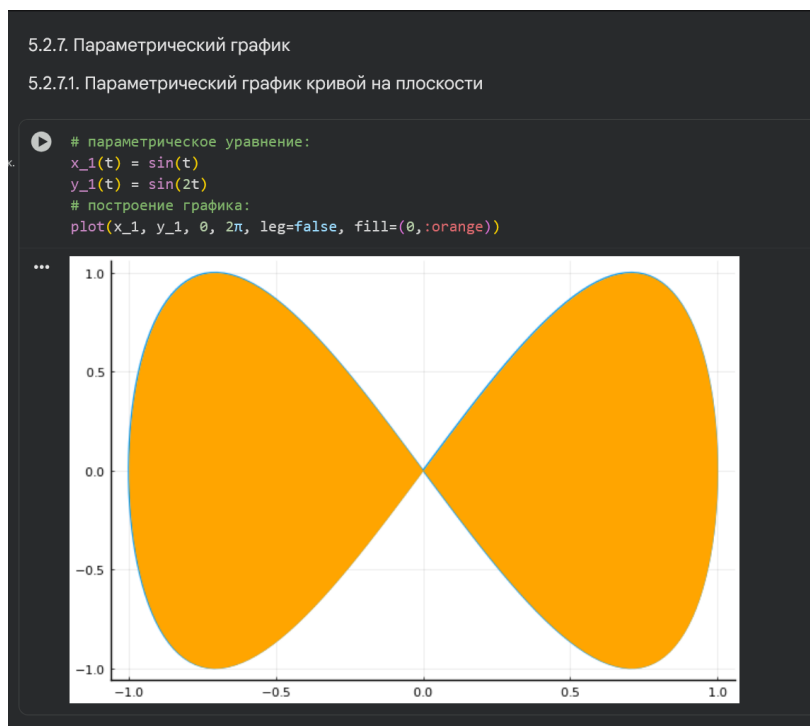


Рисунок 4.15: Параметрический график кривой на плоскости

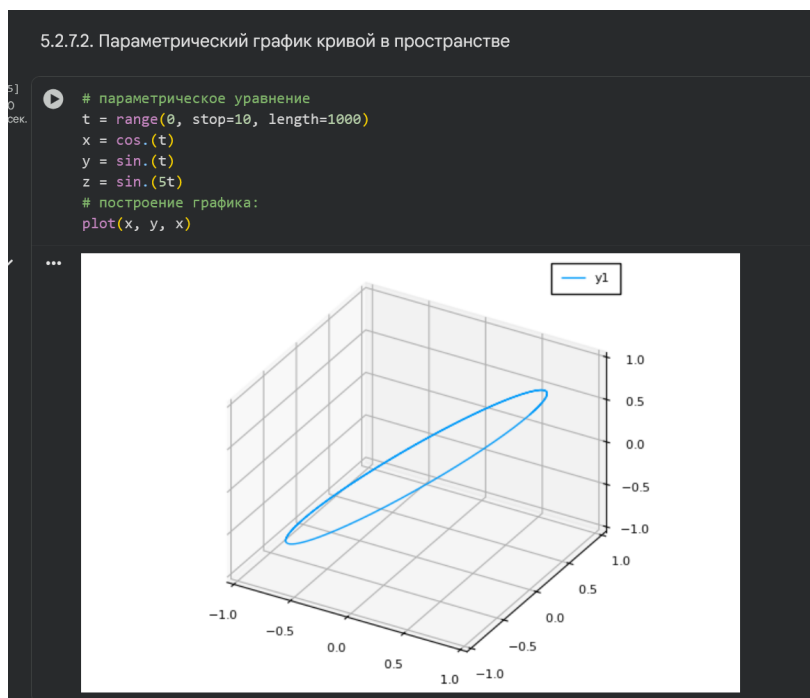


Рисунок 4.16: Параметрический график кривой в пространстве

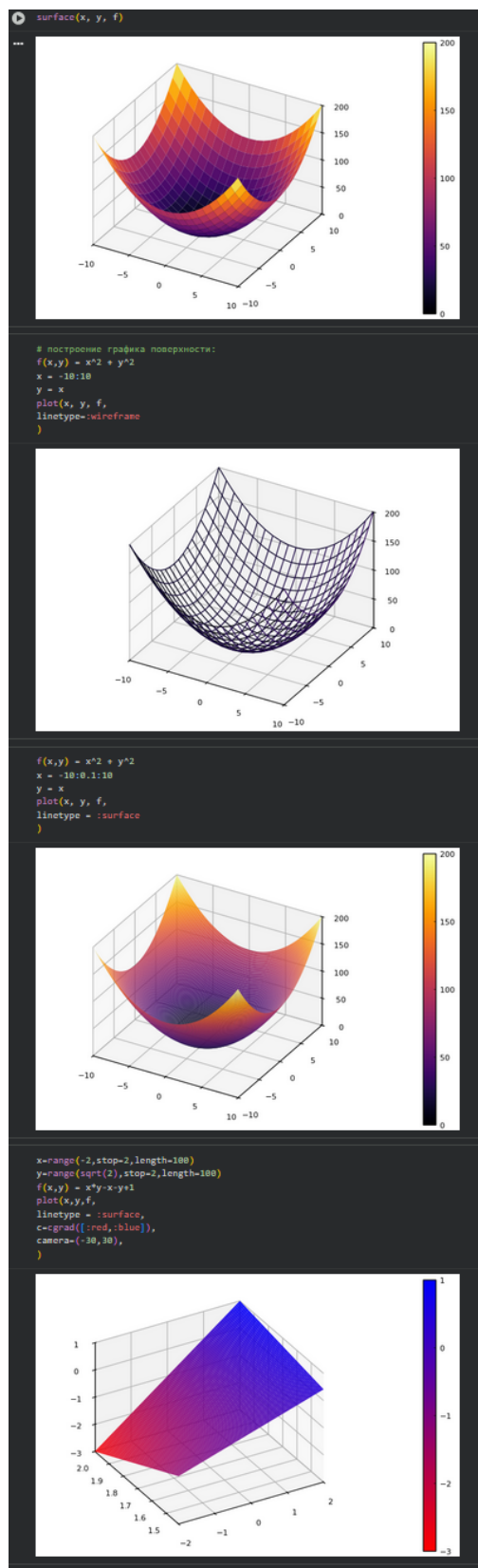
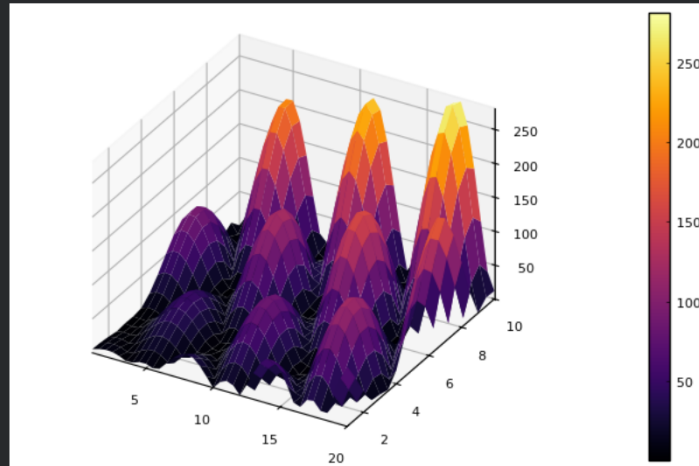


Рисунок 4.17: Параметрический график кривой в пространстве

5.2.9. Линии уровня

```
x = 1:0.5:20  
y = 1:0.5:10  
g_1(x, y) = (3x + y ^ 2) * abs(sin(x) + cos(y))  
plot(x,y,g_1,  
linetype = :surface,  
)
```



```
contour(x, y, g_1)
```



Рисунок 4.18: Линии уровня

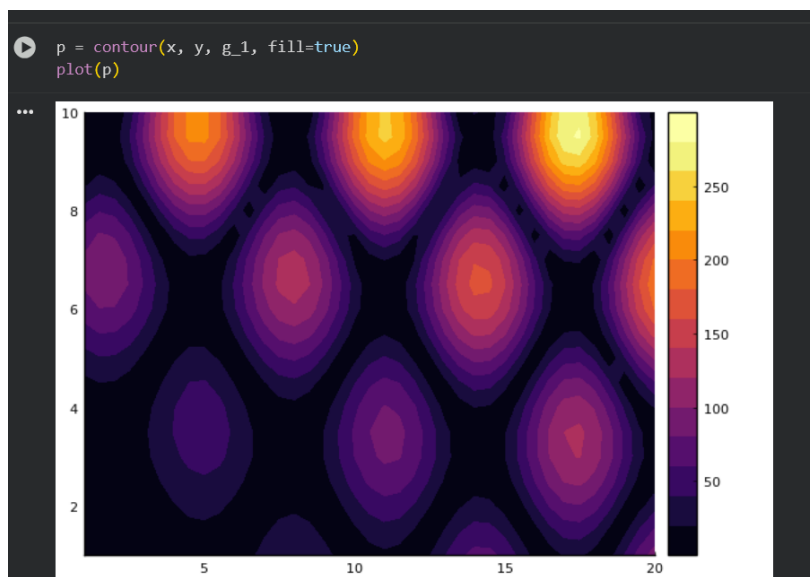


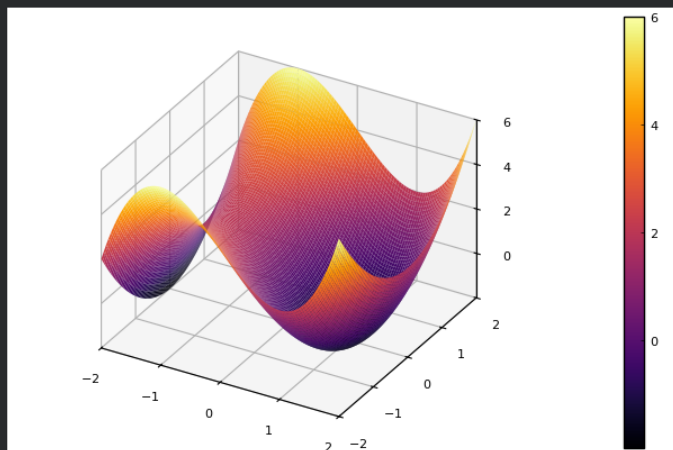
Рисунок 4.19: Линии уровня

5.2.10. Векторные поля

[48]

2
СВК.

```
# определение переменных:
X = range(-2, stop=2, length=100)
Y = range(-2, stop=2, length=100)
# определение функции:
h(x, y) = x^3 - 3x + y^2
# построение поверхности:
plot(X, Y, h,
      linestyle = :surface
    )
```



[49]

0
СВК.

```
# построение линий уровня:
contour(X, Y, h)
```

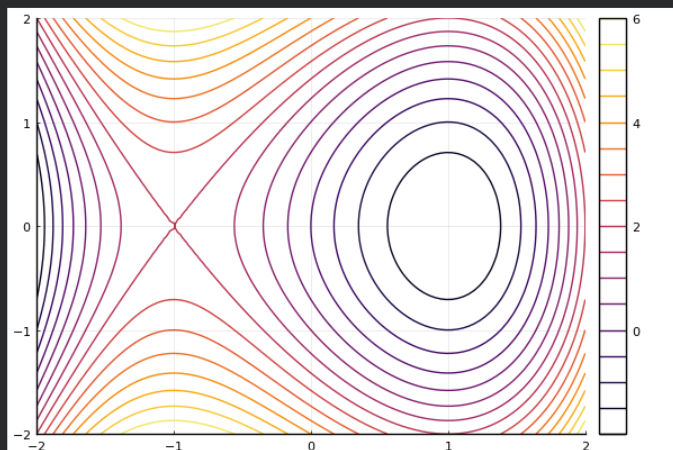


Рисунок 4.20: Векторные поля

5.2.11. Анимация

5.2.11.1. Gif-анимация

```
# построение поверхности:  
i = 0  
X = Y = range(-5,stop=5,length=40)  
Plots.surface(X, Y, (x,y) -> sin(x+10sin(i))+cos(y))  
# анимация:  
X = Y = range(-5,stop=5,length=40)  
@gif for i in range(0,stop=2π,length=100)  
    Plots.surface(X, Y, (x,y) -> sin(x+10sin(i))+cos(y))  
end
```

[Info: Saved animation to /content/tmp.gif

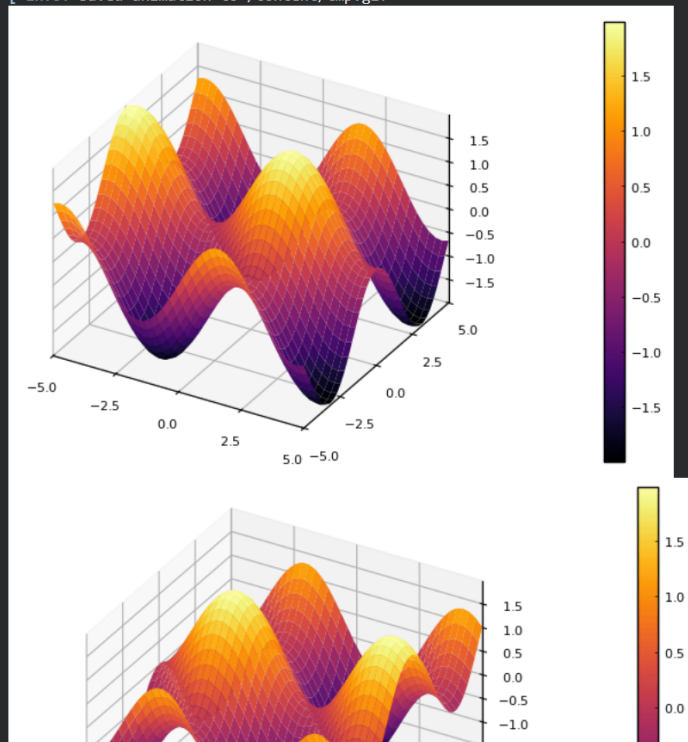


Рисунок 4.21: Gif-анимация

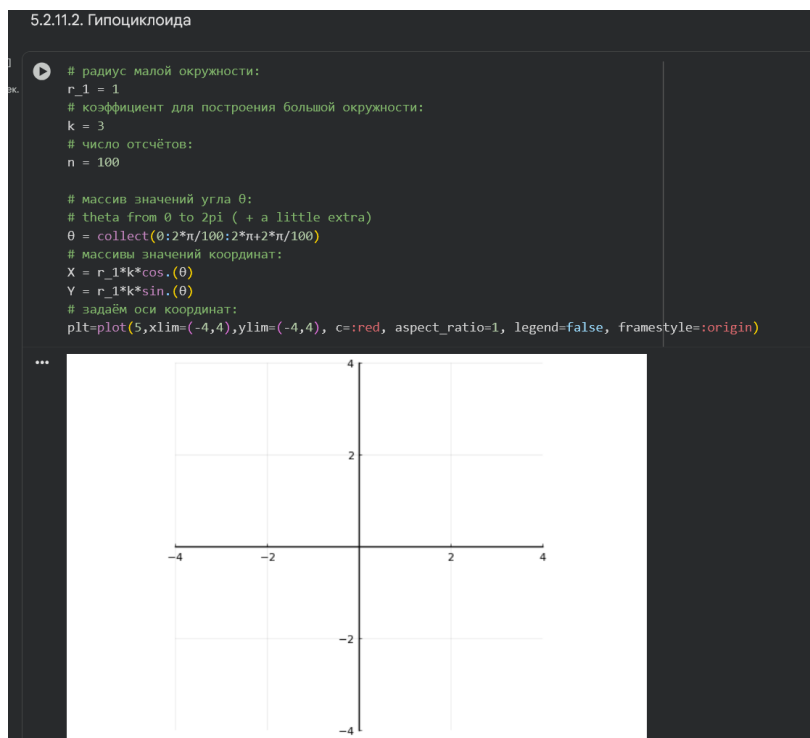


Рисунок 4.22: Гипоциклоида

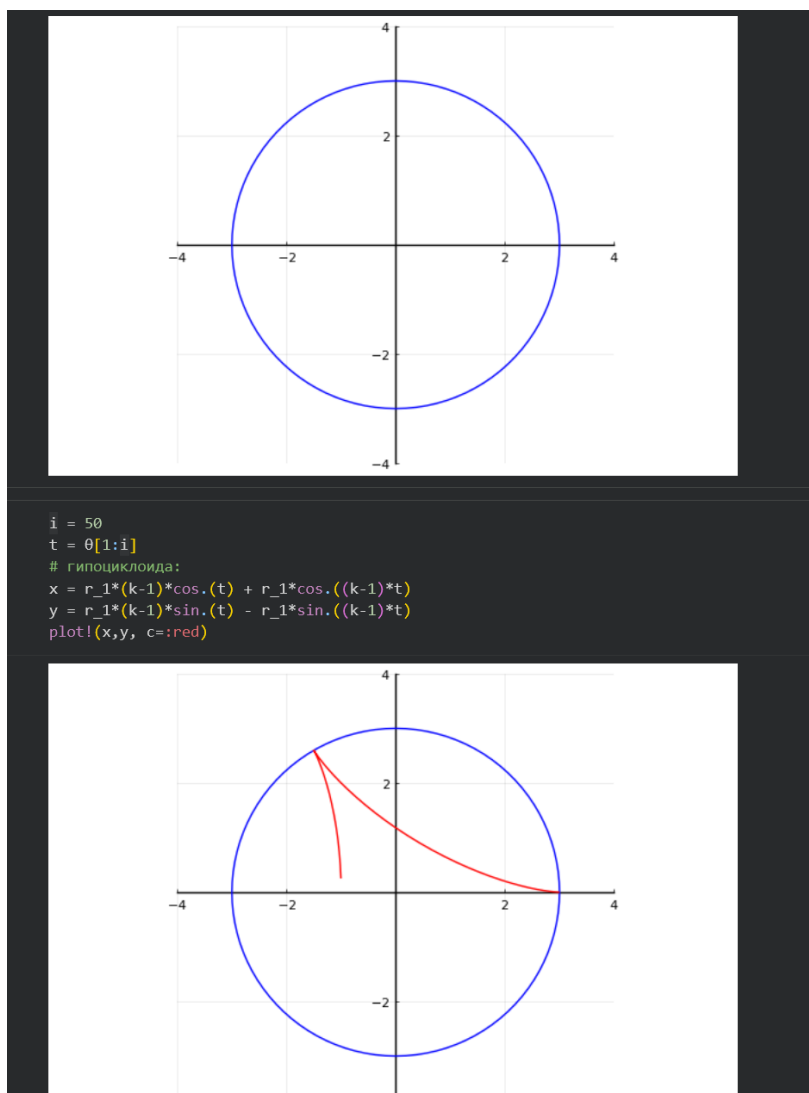
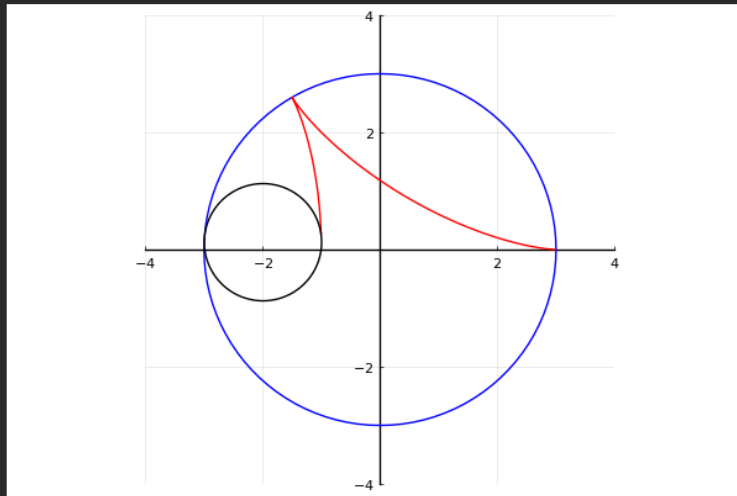


Рисунок 4.23: Гипоциклоида

```
# малая окружность:
xc = r_1*(k-1)*cos(t[end]) .+ r_1*cos.(θ)
yc = r_1*(k-1)*sin(t[end]) .+ r_1*sin.(θ)
plot!(xc,yc,c=:black)
```



```
# радиус малой окружности:
x1 = transpose([r_1*(k-1)*cos(t[end]) x[end]])
y1 = transpose([r_1*(k-1)*sin(t[end]) y[end]])
plot!(x1,y1,markershape=:circle,markersize=4,c=:black)
scatter!([x[end]],[y[end]],c=:red, markerstrokecolor=:red)
```

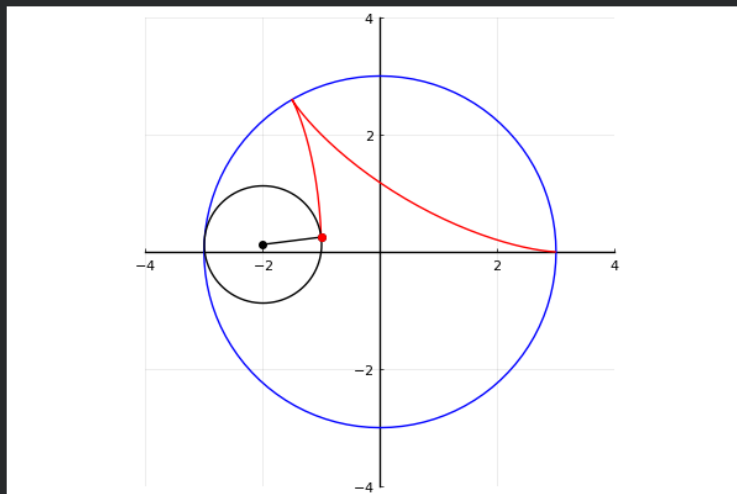


Рисунок 4.24: Гипоциклоида

5.2.12. Errorbars

```
# подключение пакета Statistics:
import Pkg
Pkg.add("Statistics")
using Statistics
sds = [1, 1/2, 1/4, 1/8, 1/16, 1/32]

n = 10
y = [mean(sd*randn(n)) for sd in sds]
errs = 1.96 * sds / sqrt(n)

Plots.plot(y,
  ylims = (-1,1),
)

Plots.plot(y,
  ylims = (-1,1),
  err = errs
)
```

Resolving package versions...
 Updating `~/.julia/environments/v1.11/Project.toml`
 [10745b16] + Statistics v1.11.1
 No Changes to `~/.julia/environments/v1.11/Manifest.toml`

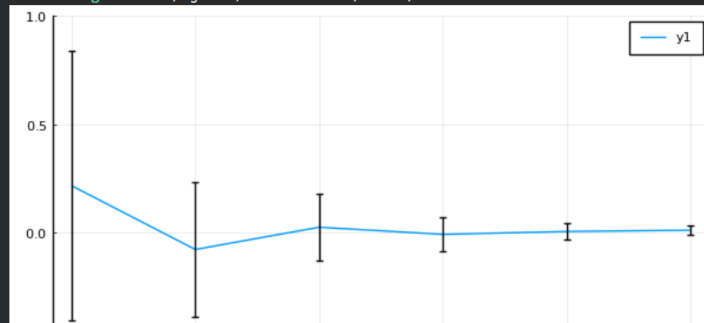


Рисунок 4.25: Errorbars



Рисунок 4.26: Errorbars

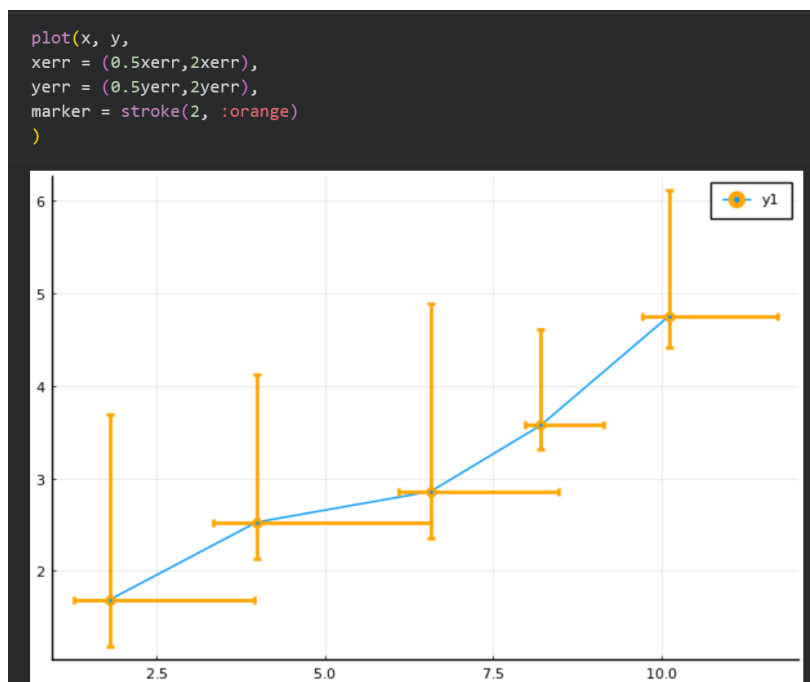


Рисунок 4.27: Errorbars



Рисунок 4.28: Использование пакета Distributions

5.2.14. Подграфики

```
# подгружаем pyplot():  
pyplot()  
# построение серии графиков:  
x=range(-2,2,length=10)  
y = rand(10,4)  
plot(x,y,layout=(4,1))
```

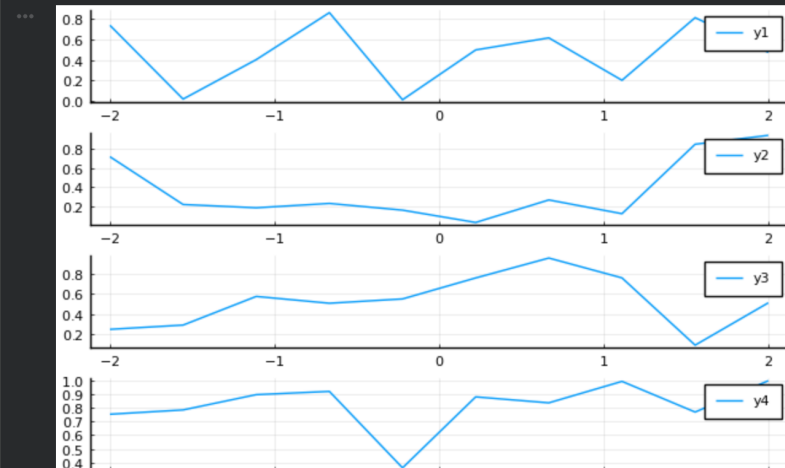


Рисунок 4.29: Подграфики

```
plot(x,y,  
layout=4  
)
```

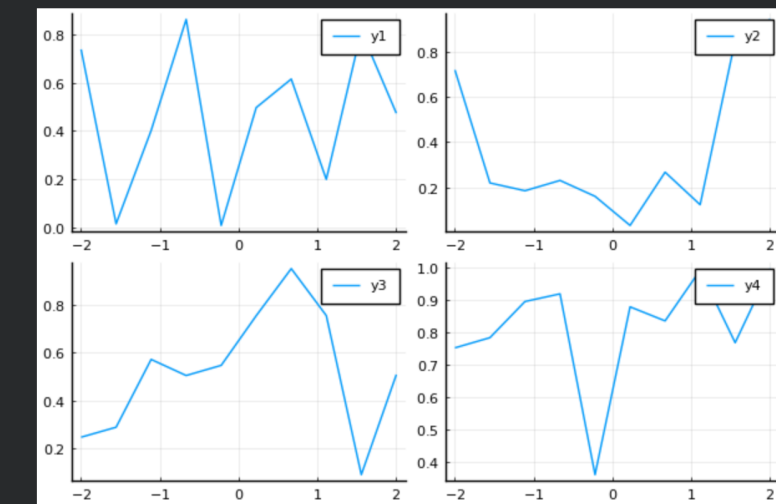


Рисунок 4.30: Подграфики



Рисунок 4.31: Подграфики

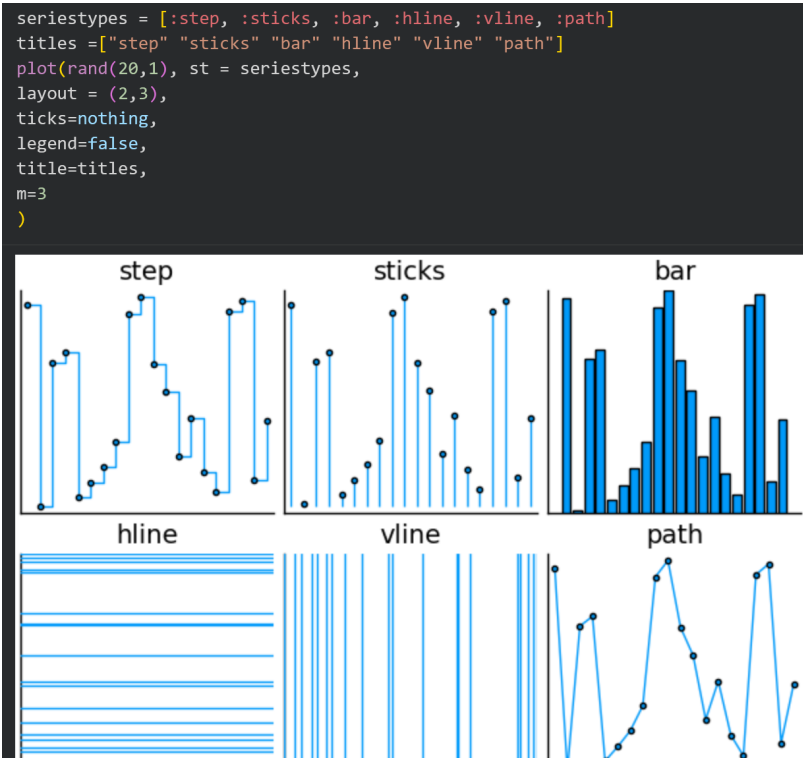


Рисунок 4.32: Подграфики



Рисунок 4.33: Подграфики

4.2 Задания для самостоятельной работы

Выполним задания (рис. 4.34-4.44).

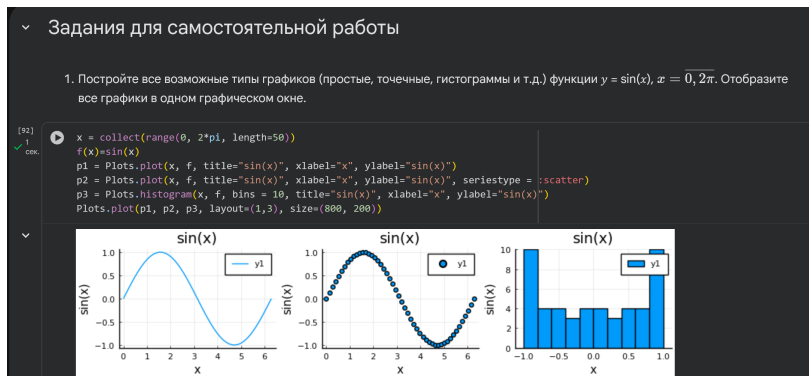


Рисунок 4.34: Задание №1

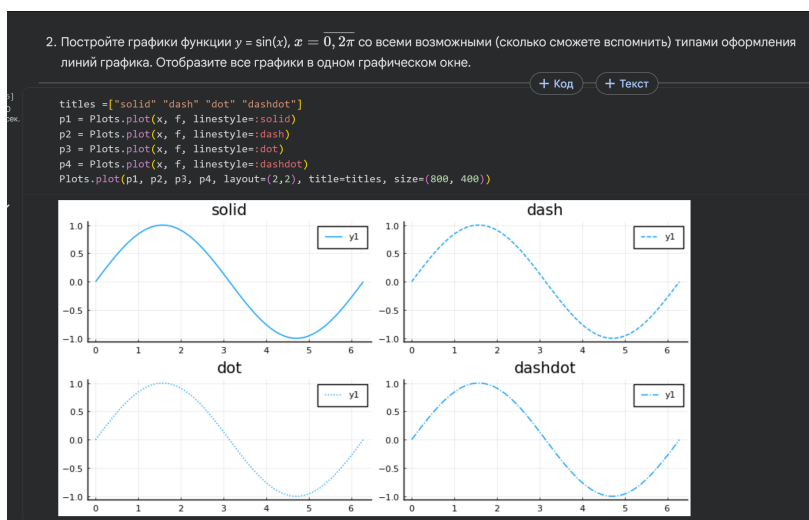


Рисунок 4.35: Задание №2

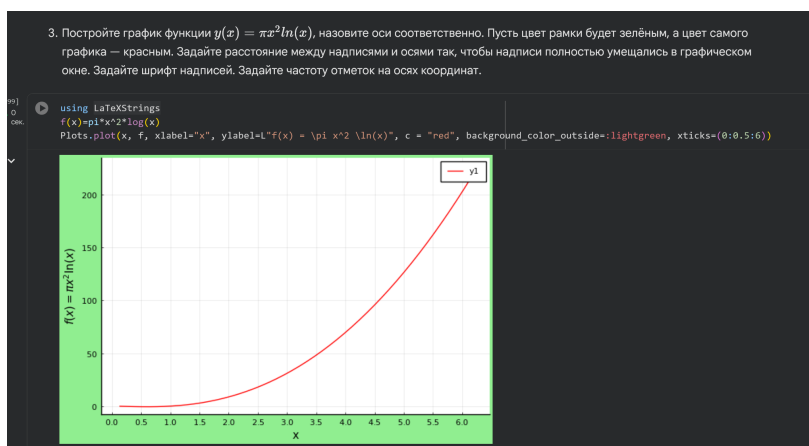


Рисунок 4.36: Задание №3

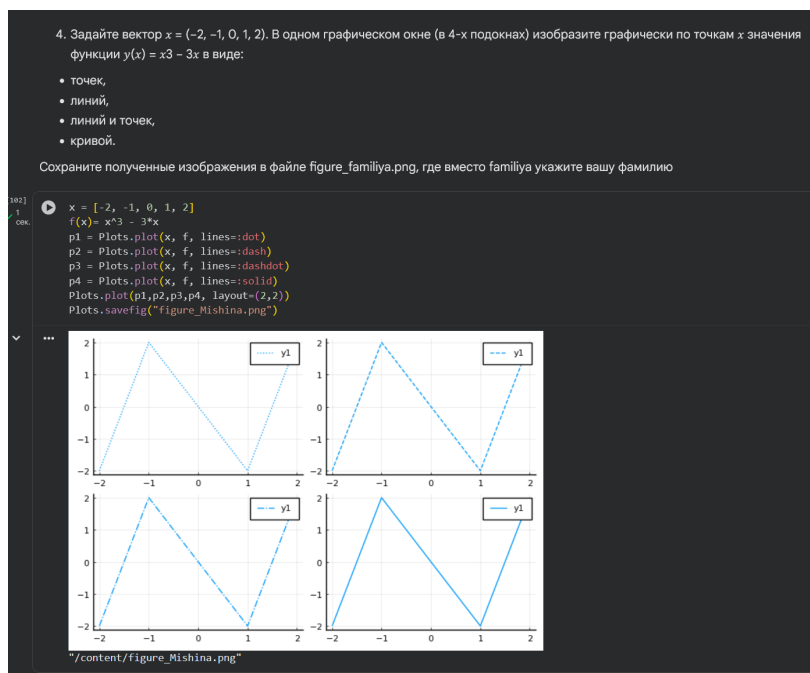


Рисунок 4.37: Задание №4

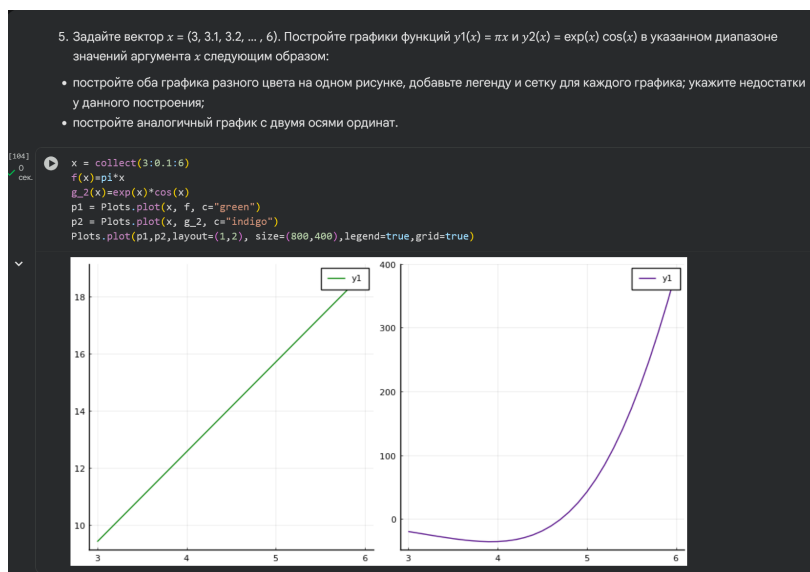


Рисунок 4.38: Задание №5

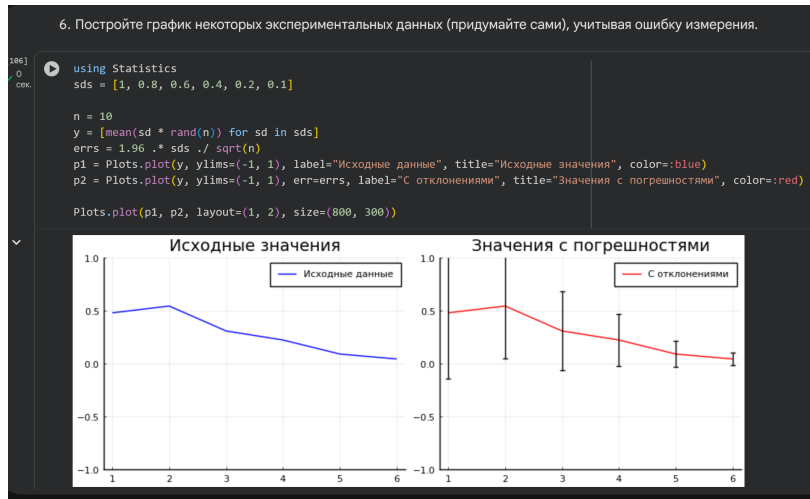


Рисунок 4.39: Задание №6

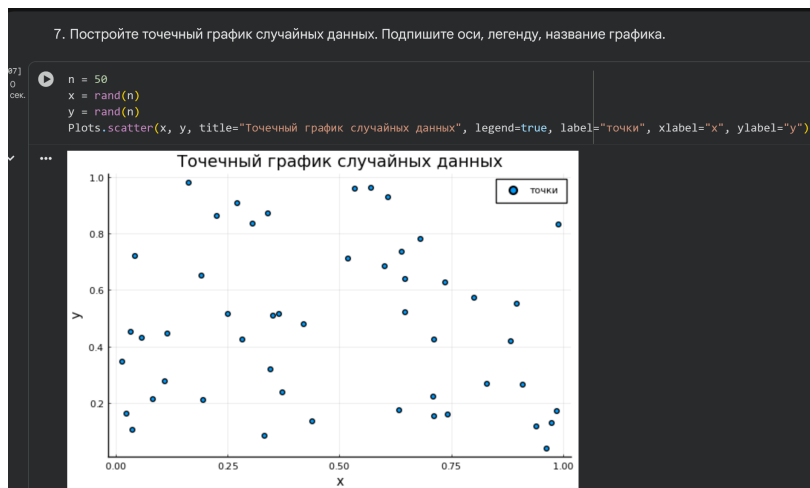


Рисунок 4.40: Задание №7



Рисунок 4.41: Задание №8

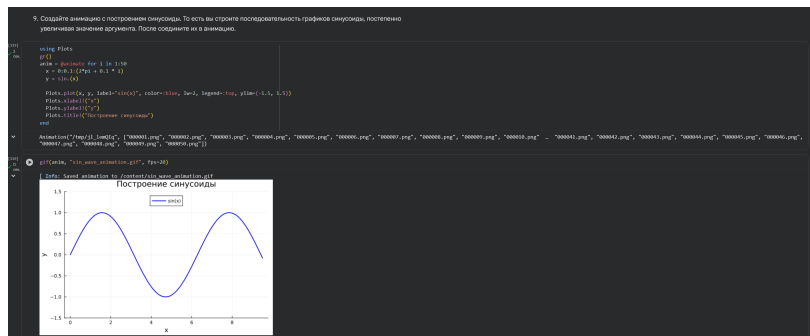


Рисунок 4.42: Задание №9

10. Постройте анимированную гипоциклоиду для 2 целых значений модуля k и 2 рациональных значений модуля k .

```

using Plots
# Параметры
r_ = 1 # Радиус малой окружности
k = 5 # Значения модуля k
n = 100
θ = collect(0:2*pi/100:2*pi+2*pi/100) # Угловые значения для построения
# Большая окружность
X = r_ * k * cos.(θ)
Y = r_ * k * sin.(θ)
anim = @animate for i in 1:n
    # задаём оси координат:
    plt = plot(xlim=(-6,6), ylim=(-6,6), c=:white, aspect_ratio=1, legend=false, framestyle=:origin)
    # большая окружность:
    plot!(plt, X, Y, c=:blue, legend=false)
    t = θ[i:i]
    # гипоциклоида:
    x = r_*(k-1)*cos.(t) .+ r_*cos.((k-1).*t)
    y = r_*(k-1)*sin.(t) .- r_*sin.((k-1).*t)
    plot!(plt, x, y, c=:red, linewidth=2)
    # малая окружность:
    xc = r_*(k-1)*cos(t[end]) .+ r_*cos.(θ)
    yc = r_*(k-1)*sin(t[end]) .+ r_*sin.(θ)
    plot!(plt, xc, yc, c=:black, linewidth=1)
    # радиус малой окружности:
    xl = [r_*(k-1)*cos(t[end]), x[end]]
    yl = [r_*(k-1)*sin(t[end]), y[end]]
    plot!(plt, xl, yl, c=:black, linewidth=2)
    # точка на гипоциклоиде:
    scatter!(plt, [x[end]], [y[end]], c=:red, markersize=6, markerstrokecolor=:red)
    # центр малой окружности:
    scatter!(plt, [r_*(k-1)*cos(t[end])], [r_*(k-1)*sin(t[end])], c=:black, markersize=4, markerstrokecolor=:black)
end
# Сохраним анимацию в gif-файл:
gif(anim, "hypocycloid.gif", fps=20)

```

... [Info: Saved animation to /content/hypocycloid.gif

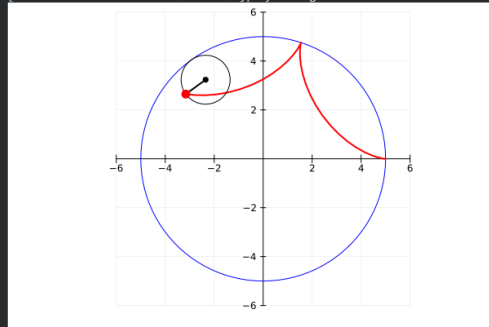


Рисунок 4.43: Задание №10

11. Постройте анимированную эпициклоиду для 2 целых значений модуля k и 2 рациональных значений модуля k .

```
using Plots

function create_epicycloid_animation(k, r=1, n=100)
    θ = collect(0:2π/100:4π) # Увеличиваем диапазон для эпициклоиды

    anim = @animate for i in 1:n
        # Большая (неподвижная) окружность
        R_big = r # Радиус большой окружности
        X_big = R_big * cos(θ)
        Y_big = R_big * sin(θ)

        plt = plot(xlim=(-4*(k+1), 4*(k+1)), ylim=(-4*(k+1), 4*(k+1)), c=:white,
            aspect_ratio=1, legend=false, framestyle=:origin,
            title="Эпициклоида, k = $k")

        # Большая неподвижная окружности (основа)
        plot!(plt, X_big, Y_big, c=:blue, linewidth=2, label="Большая окружность")

        t = θ[1:i]

        # Эпициклоида
        x = r*(k+1)*cos.(t) .- r*cos.(k+1).*t
        y = r*(k+1)*sin.(t) .- r*sin.(k+1).*t
        plot!(plt, x, y, c=:red, linewidth=3, label="Эпициклоида")

        # Катящаяся малая окружность
        current_angle = t[end]
        R_small = r * k # Радиус малой окружности
        X_small = r*(k+1)*cos(current_angle) .+ R_small*cos(θ)
        Y_small = r*(k+1)*sin(current_angle) .+ R_small*sin(θ)
        plot!(plt, X_small, Y_small, c=:green, linewidth=1, label="Малая окружность")

        # Точка на эпициклоиде
        scatter!(plt, [x[end]], [y[end]], c=:red, markersize=8, label="Точка")

        # Центр катящейся малой окружности
        scatter!(plt, [r*(k+1)*cos(current_angle)], [r*(k+1)*sin(current_angle)],
            c=:black, markersize=5, label="Центр малой окружности")

        # Линия от центра малой окружности до точки на эпициклоиде
        plot!(plt, [r*(k+1)*cos(current_angle), x[end]],
            [r*(k+1)*sin(current_angle), y[end]],
            c=:black, linewidth=2, label="Радиус")

    end

    return anim
end

# 2 целых значения k
k1 = 2
k2 = 5

# 2 рациональных значения k
k3 = 3//2
k4 = 5//2

# Создаем анимации
println("Создание анимации для k = $k1...")
anim1 = create_epicycloid_animation(k1)
gif(anim1, "epicycloid_k1.gif", fps=20)

println("Создание анимации для k = $k2...")
anim2 = create_epicycloid_animation(k2)
gif(anim2, "epicycloid_k2.gif", fps=20)

println("Создание анимации для k = $k3...")
anim3 = create_epicycloid_animation(k3)
gif(anim3, "epicycloid_k3.gif", fps=20)

println("Создание анимации для k = $k4...")
anim4 = create_epicycloid_animation(k4)
gif(anim4, "epicycloid_k4.gif", fps=20)
```

```
Создание анимации для k = 2...
Создание анимации для k = 5...
Создание анимации для k = 3//2...
Создание анимации для k = 5//2...
[ Info: Saved animation to /content/epicycloid_k1.gif
[ Info: Saved animation to /content/epicycloid_k2.gif
[ Info: Saved animation to /content/epicycloid_k3.gif
[ Info: Saved animation to /content/epicycloid_k4.gif
```

Эпициклоида, $k = 5//2$

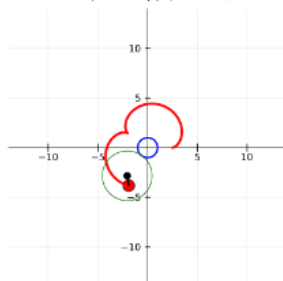


Рисунок 4.44: Задание №11

5 Выводы

В результате выполнения данной лабораторной работы я освоил синтаксис языка Julia для построения графиков.

Список литературы